

Untitled

March 5, 2026

```
[1]: import pandas as pd
from matplotlib import pyplot as plt
import numpy as np
import seaborn as sns
import glob
import os
import re
```

```
[7]: IGS_path = r"C:
      ↪\Users\jabba\Desktop\Code\machine_learning\AUC_mastercard_challenge\src\Inclusive_Growth_Sc
      ↪csv"
```

```
IGS = pd.read_csv(IGS_path)
IGS.head()

# Select all rows, and all columns from index 1 onwards
IGS_filtered = IGS.iloc[:, 1:]

# 1. Get the values of the first row and assign them as new column names
IGS_filtered.columns = IGS_filtered.iloc[0].values

# 2. Drop the first row (which is now a duplicate of the header)
IGS_filtered = IGS_filtered.drop(IGS_filtered.index[0])

# Optional: Reset the index of the DataFrame for cleaner indexing
IGS_filtered = IGS_filtered.reset_index(drop=True)

IGS_filtered = IGS_filtered.drop(IGS_filtered.index[0])

IGS_filtered.head(10)
```

C:\Users\jabba\AppData\Local\Temp\ipykernel_17904\1244400182.py:3: DtypeWarning: Columns (2,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,22,23,24,25,26,27,28,29,30,31,32,33,34,35,36,37,38,39,40,41,42,43,44,45,46,47,48,49,50,51,52,53,54,55,56,57,58,59,60,61,62,63,64,65,66,67,68,69,70,71) have mixed types. Specify dtype option on import or set low_memory=False.

```
IGS = pd.read_csv(IGS_path)
```

```

[7]: Is an Opportunity Zone Census Tract FIPS code      County      State \
1      NaN      01097002302 Mobile County Alabama
2      NaN      01097002302 Mobile County Alabama
3      NaN      01097002302 Mobile County Alabama
4      NaN      01097002302 Mobile County Alabama
5      NaN      01097002302 Mobile County Alabama
6      NaN      01097002302 Mobile County Alabama
7      NaN      01097002302 Mobile County Alabama
8      NaN      01097002302 Mobile County Alabama
9      NaN      01097002302 Mobile County Alabama
10     NaN      01097003404 Mobile County Alabama

```

```

      Year Inclusive Growth Score Growth Inclusion Place Place Growth ... \
1  2017      23      26      20      24      18 ...
2  2018      28      35      20      24      12 ...
3  2019      22      22      22      26      16 ...
4  2020      34      45      23      28      18 ...
5  2021      28      33      23      29      10 ...
6  2022      24      23      24      32      13 ...
7  2023     27.5    34.5      21      29      24 ...
8  2024      32      42      21      35      36 ...
9  2025      29      37      21      35      36 ...
10 2017      38      48      27      34      42 ...

```

```

      Female Above Poverty Tract, % Gini Coefficient Score Gini Coefficient Base \
1      63.1      48      41.4
2      58.3      42      41.6
3      60.2      45      41.7
4      59.7      43      41.8
5      67.7      33      41.3
6      67.6      8      41.4
7      70      10      41.5
8      70      10      41.5
9      70      10      41.5
10     61.8      59      41.4

```

```

      Gini Coefficient Tract Early Education Enrollment Score \
1      41.7      14.1
2      42.8      15.1
3      42.4      23.1
4      42.7      41.1
5      44.2      NaN
6      51.9      29.1
7      50.7      70
8      50.7      70
9      50.7      70
10     40.2      64

```

	Early Education Enrollment Base, %	Early Education Enrollment Tract, % \
1	23.1	10.5
2	22.9	10.9
3	22.9	13.9
4	23	20.2
5	25.3	NaN
6	25.2	17.7
7	25.3	34.9
8	25.3	34.9
9	25.3	34.9
10	23.1	28.4

	Health Insurance Coverage Score	Health Insurance Coverage Base, % \
1	17	90
2	19	91.2
3	15	92.3
4	13	93.2
5	10	94.1
6	25	94
7	14	93.9
8	14	93.9
9	14	93.9
10	8	90

	Health Insurance Coverage Tract, %
1	83.1
2	85.2
3	85.2
4	85.6
5	84.2
6	89.7
7	86.1
8	86.1
9	86.1
10	79.1

[10 rows x 71 columns]

```
[3]: # =====
# 0) Setup
# =====
import os
import re
import time
from pathlib import Path
import requests
```

```

import pandas as pd
import numpy as np

DATA_DIR = Path("data")
RAW_DIR = DATA_DIR / "raw"
OUT_DIR = DATA_DIR / "processed"
RAW_DIR.mkdir(parents=True, exist_ok=True)
OUT_DIR.mkdir(parents=True, exist_ok=True)

# CENSUS_API_KEY = os.getenv("", "") # set in env, or paste below
CENSUS_API_KEY = "d508dd731af7f9d40a3952ee423d91a8124c242e"

YEARS = list(range(2017, 2023)) # adjust to match IGS years in your file

# =====
# 1) Load + Clean IGS (handles messy header rows)
# =====
def _find_header_row(df_raw: pd.DataFrame, required_tokens=("Census Tract",
↳FIPS", "Year")) -> int:
    """
    Finds the row index that looks like the real header row.
    We look for cells containing key tokens (case-insensitive).
    """
    tokens = [t.lower() for t in required_tokens]
    for i in range(min(len(df_raw), 50)): # scan first 50 rows
        row = df_raw.iloc[i].astype(str).str.lower().tolist()
        if all(any(tok in cell for cell in row) for tok in tokens):
            return i
        raise ValueError("Could not find header row. Check the raw IGS file_
↳formatting.")

def load_igs_any(path: str) -> pd.DataFrame:
    p = Path(path)
    if p.suffix.lower() in [".xlsx", ".xls"]:
        raw = pd.read_excel(p, header=None)
    else:
        raw = pd.read_csv(p, header=None)

    hdr_i = _find_header_row(raw, required_tokens=("Census Tract FIPS", "Year"))
    header = raw.iloc[hdr_i].tolist()
    df = raw.iloc[hdr_i + 1 :].copy()
    df.columns = header
    df = df.dropna(how="all")

    # standardize key columns
    # (keep original names too if you want; this just normalizes)

```

```

col_map = {}
for c in df.columns:
    if isinstance(c, str):
        c2 = c.strip()
        col_map[c] = c2
df = df.rename(columns=col_map)

# Find the likely key columns
# (your dataset uses these exact names from your column list)
df["geoid"] = df["Census Tract FIPS code"].astype(str).str.replace(r"\.0$", "",
↪", regex=True).str.zfill(11)
df["year"] = pd.to_numeric(df["Year"], errors="coerce").astype("Int64")

# Normalize target cols
# You can change target to pillars later; start with Inclusive Growth Score.
df["igs_total"] = pd.to_numeric(df.get("Inclusive Growth Score", np.nan),
↪errors="coerce")

# keep useful IGS columns
keep = [
    "geoid", "year",
    "igs_total",
    "Place", "Economy", "Community", "Inclusion", "Growth",
    "URBAN CODE", "Is an Opportunity Zone", "BENCHMARK"
]
existing = [c for c in keep if c in df.columns]
df = df[existing].copy()

# clean types
if "URBAN CODE" in df.columns:
    df["urban_code"] = pd.to_numeric(df["URBAN CODE"], errors="coerce")
if "Is an Opportunity Zone" in df.columns:
    df["is_opp_zone"] = pd.to_numeric(df["Is an Opportunity Zone"],
↪errors="coerce")
if "BENCHMARK" in df.columns:
    df["benchmark"] = df["BENCHMARK"].astype(str)

df = df.drop(columns=[c for c in ["URBAN CODE", "Is an Opportunity Zone",
↪"BENCHMARK"] if c in df.columns])

# Drop rows missing keys
df = df.dropna(subset=["geoid", "year"])
df["year"] = df["year"].astype(int)

return df

```

```

# =====
# 2) Census API helpers (ACS 5-year)
# =====
def chunks(lst, n):
    for i in range(0, len(lst), n):
        yield lst[i:i+n]

def get_state_fips(year: int, api_key: str) -> list[str]:
    url = f"https://api.census.gov/data/{year}/acs/acs5"
    params = {"get": "NAME", "for": "state:*", "key": api_key}
    r = requests.get(url, params=params, timeout=60)
    r.raise_for_status()
    data = r.json()
    header, rows = data[0], data[1:]
    df = pd.DataFrame(rows, columns=header)
    return sorted(df["state"].unique().tolist())

def census_get_tracts(year: int, state_fips: str, vars_: list[str], api_key: str) -> pd.DataFrame:
    """
    Pulls ACS 5-year data for ALL tracts in ALL counties of a state.
    Uses the query shape shown in Census API examples:
        ...?get=...&for=tract:*&in=state:06&in=county:* (wildcards supported)
    """
    base = f"https://api.census.gov/data/{year}/acs/acs5"
    get_str = "NAME," + ",".join(vars_)

    # requests needs duplicate 'in' keys -> use list of tuples
    params = [
        ("get", get_str),
        ("for", "tract:*"),
        ("in", f"state:{state_fips}"),
        ("in", "county:*"),
        ("key", api_key),
    ]

    r = requests.get(base, params=params, timeout=120)
    r.raise_for_status()
    data = r.json()
    header, rows = data[0], data[1:]
    df = pd.DataFrame(rows, columns=header)

    df["year"] = year
    return df

def to_numeric_safe(df: pd.DataFrame, exclude=("NAME", "state", "county", "tract", "year")) -> pd.DataFrame:

```

```

for c in df.columns:
    if c in exclude:
        continue
    df[c] = pd.to_numeric(df[c], errors="coerce")
return df

# =====
# 3) Define the ACS variables we need (tables/metrics)
# =====

ACS_VARS = [
    # --- population / age (B01001) ---
    "B01001_001E", # total pop
    "B01001_003E", "B01001_004E", "B01001_005E", "B01001_006E", # male <18
    ↪ groups
    "B01001_027E", "B01001_028E", "B01001_029E", "B01001_030E", # female <18
    ↪ groups

    # --- race (B02001) ---
    "B02001_001E", "B02001_002E", "B02001_003E", "B02001_005E", "B02001_008E",

    # --- education (B15003) BA+ ---
    "B15003_001E", "B15003_022E", "B15003_023E", "B15003_024E", "B15003_025E",

    # --- employment (B23025) ---
    "B23025_001E", "B23025_002E", "B23025_003E", "B23025_004E", "B23025_005E",
    ↪ "B23025_007E",

    # --- commute time (B08303) ---
    "B08303_001E", "B08303_002E", "B08303_003E", "B08303_004E", "B08303_005E",
    ↪ "B08303_006E", "B08303_007E", "B08303_008E",

    # --- internet subscription (B28002) ---
    "B28002_001E", "B28002_002E",

    # --- rent burden (B25070) ---
    "B25070_001E", "B25070_002E", "B25070_003E", "B25070_004E", "B25070_005E",
    ↪ "B25070_006E",

    # --- owner cost burden (B25091) ---
    "B25091_001E",
    "B25091_003E", "B25091_004E", "B25091_005E", "B25091_006E", "B25091_007E",
    ↪ # with mortgage <30%
    "B25091_014E", "B25091_015E", "B25091_016E", "B25091_017E", "B25091_018E",
    ↪ # without mortgage <30%

```

```

# --- occupancy (B25002) ---
"B25002_001E", "B25002_002E",

# --- home value (B25077) ---
"B25077_001E",

# --- female above poverty (B17001) ---
"B17001_017E", "B17001_046E",

# --- gini (B19083) ---
"B19083_001E",

# --- per-capita income (B19301) ---
"B19301_001E",

# --- early education enrollment proxy (B14003) (3-4 years enrolled public/
↳private, male/female) ---
"B14003_004E", "B14003_013E", "B14003_032E", "B14003_041E",

# --- health insurance (B27001) total + uninsured-by-sex/age blocks ---
"B27001_001E",
# male uninsured (patterned)
↳
↳"B27001_005E", "B27001_008E", "B27001_011E", "B27001_014E", "B27001_017E", "B27001_020E", "B27001_023E", "B27001_026E", "B27001_029E", "B27001_032E", "B27001_035E", "B27001_038E", "B27001_041E", "B27001_044E", "B27001_047E", "B27001_050E", "B27001_053E", "B27001_056E", "B27001_059E", "B27001_062E", "B27001_065E", "B27001_068E", "B27001_071E", "B27001_074E", "B27001_077E", "B27001_080E", "B27001_083E", "B27001_086E", "B27001_089E", "B27001_092E", "B27001_095E", "B27001_098E", "B27001_101E", "B27001_104E", "B27001_107E", "B27001_110E", "B27001_113E", "B27001_116E", "B27001_119E", "B27001_122E", "B27001_125E", "B27001_128E", "B27001_131E", "B27001_134E", "B27001_137E", "B27001_140E", "B27001_143E", "B27001_146E", "B27001_149E", "B27001_152E", "B27001_155E", "B27001_158E", "B27001_161E", "B27001_164E", "B27001_167E", "B27001_170E", "B27001_173E", "B27001_176E", "B27001_179E", "B27001_182E", "B27001_185E", "B27001_188E", "B27001_191E", "B27001_194E", "B27001_197E", "B27001_200E", "B27001_203E", "B27001_206E", "B27001_209E", "B27001_212E", "B27001_215E", "B27001_218E", "B27001_221E", "B27001_224E", "B27001_227E", "B27001_230E", "B27001_233E", "B27001_236E", "B27001_239E", "B27001_242E", "B27001_245E", "B27001_248E", "B27001_251E", "B27001_254E", "B27001_257E", "B27001_260E", "B27001_263E", "B27001_266E", "B27001_269E", "B27001_272E", "B27001_275E", "B27001_278E", "B27001_281E", "B27001_284E", "B27001_287E", "B27001_290E", "B27001_293E", "B27001_296E", "B27001_299E", "B27001_302E", "B27001_305E", "B27001_308E", "B27001_311E", "B27001_314E", "B27001_317E", "B27001_320E", "B27001_323E", "B27001_326E", "B27001_329E", "B27001_332E", "B27001_335E", "B27001_338E", "B27001_341E", "B27001_344E", "B27001_347E", "B27001_350E", "B27001_353E", "B27001_356E", "B27001_359E", "B27001_362E", "B27001_365E", "B27001_368E", "B27001_371E", "B27001_374E", "B27001_377E", "B27001_380E", "B27001_383E", "B27001_386E", "B27001_389E", "B27001_392E", "B27001_395E", "B27001_398E", "B27001_401E", "B27001_404E", "B27001_407E", "B27001_410E", "B27001_413E", "B27001_416E", "B27001_419E", "B27001_422E", "B27001_425E", "B27001_428E", "B27001_431E", "B27001_434E", "B27001_437E", "B27001_440E", "B27001_443E", "B27001_446E", "B27001_449E", "B27001_452E", "B27001_455E", "B27001_458E", "B27001_461E", "B27001_464E", "B27001_467E", "B27001_470E", "B27001_473E", "B27001_476E", "B27001_479E", "B27001_482E", "B27001_485E", "B27001_488E", "B27001_491E", "B27001_494E", "B27001_497E", "B27001_500E", "B27001_503E", "B27001_506E", "B27001_509E", "B27001_512E", "B27001_515E", "B27001_518E", "B27001_521E", "B27001_524E", "B27001_527E", "B27001_530E", "B27001_533E", "B27001_536E", "B27001_539E", "B27001_542E", "B27001_545E", "B27001_548E", "B27001_551E", "B27001_554E", "B27001_557E", "B27001_560E", "B27001_563E", "B27001_566E", "B27001_569E", "B27001_572E", "B27001_575E", "B27001_578E", "B27001_581E", "B27001_584E", "B27001_587E", "B27001_590E", "B27001_593E", "B27001_596E", "B27001_599E", "B27001_602E", "B27001_605E", "B27001_608E", "B27001_611E", "B27001_614E", "B27001_617E", "B27001_620E", "B27001_623E", "B27001_626E", "B27001_629E", "B27001_632E", "B27001_635E", "B27001_638E", "B27001_641E", "B27001_644E", "B27001_647E", "B27001_650E", "B27001_653E", "B27001_656E", "B27001_659E", "B27001_662E", "B27001_665E", "B27001_668E", "B27001_671E", "B27001_674E", "B27001_677E", "B27001_680E", "B27001_683E", "B27001_686E", "B27001_689E", "B27001_692E", "B27001_695E", "B27001_698E", "B27001_701E", "B27001_704E", "B27001_707E", "B27001_710E", "B27001_713E", "B27001_716E", "B27001_719E", "B27001_722E", "B27001_725E", "B27001_728E", "B27001_731E", "B27001_734E", "B27001_737E", "B27001_740E", "B27001_743E", "B27001_746E", "B27001_749E", "B27001_752E", "B27001_755E", "B27001_758E", "B27001_761E", "B27001_764E", "B27001_767E", "B27001_770E", "B27001_773E", "B27001_776E", "B27001_779E", "B27001_782E", "B27001_785E", "B27001_788E", "B27001_791E", "B27001_794E", "B27001_797E", "B27001_800E", "B27001_803E", "B27001_806E", "B27001_809E", "B27001_812E", "B27001_815E", "B27001_818E", "B27001_821E", "B27001_824E", "B27001_827E", "B27001_830E", "B27001_833E", "B27001_836E", "B27001_839E", "B27001_842E", "B27001_845E", "B27001_848E", "B27001_851E", "B27001_854E", "B27001_857E", "B27001_860E", "B27001_863E", "B27001_866E", "B27001_869E", "B27001_872E", "B27001_875E", "B27001_878E", "B27001_881E", "B27001_884E", "B27001_887E", "B27001_890E", "B27001_893E", "B27001_896E", "B27001_899E", "B27001_902E", "B27001_905E", "B27001_908E", "B27001_911E", "B27001_914E", "B27001_917E", "B27001_920E", "B27001_923E", "B27001_926E", "B27001_929E", "B27001_932E", "B27001_935E", "B27001_938E", "B27001_941E", "B27001_944E", "B27001_947E", "B27001_950E", "B27001_953E", "B27001_956E", "B27001_959E", "B27001_962E", "B27001_965E", "B27001_968E", "B27001_971E", "B27001_974E", "B27001_977E", "B27001_980E", "B27001_983E", "B27001_986E", "B27001_989E", "B27001_992E", "B27001_995E", "B27001_998E", "B27001_1000E",
]

# =====
# 4) Download ACS for all states + years (cached)
# =====
def download_acs_all(years: list[int], api_key: str, max_vars_per_call: int = 45) -> pd.DataFrame:
    if not api_key:
        raise ValueError("Missing CENSUS_API_KEY. Set env var CENSUS_API_KEY or paste it into the notebook.")

    all_parts = []
    for year in years:
        year_out = RAW_DIR / f"acs5_tract_{year}.parquet"
        if year_out.exists():
            print(f"[cache] {year_out}")
            all_parts.append(pd.read_parquet(year_out))
            continue

```



```

states = get_state_fips(year, api_key)
print(f"[year {year}] states={len(states)}")

state_frames = []
for st in states:
    merged = None
    for var_chunk in chunks(ACS_VARS, max_vars_per_call):
        df_chunk = census_get_tracts(year, st, var_chunk, api_key)
        df_chunk = to_numeric_safe(df_chunk)

        keys = ["state", "county", "tract", "NAME", "year"]
        if merged is None:
            merged = df_chunk
        else:
            merged = merged.merge(df_chunk, on=keys, how="outer")

        time.sleep(0.25) # be polite to the API

    state_frames.append(merged)

df_year = pd.concat(state_frames, ignore_index=True)
df_year.to_parquet(year_out, index=False)
all_parts.append(df_year)

return pd.concat(all_parts, ignore_index=True)

# =====
# 5) Feature engineering to mirror IGS-style public proxies
# =====
def safe_div(n, d):
    return np.where((d is None) | (pd.isna(d)) | (d == 0), np.nan, n / d)

def make_features(acs: pd.DataFrame) -> pd.DataFrame:
    # GEOID
    acs["state"] = acs["state"].astype(str).str.zfill(2)
    acs["county"] = acs["county"].astype(str).str.zfill(3)
    acs["tract"] = acs["tract"].astype(str).str.zfill(6)
    acs["geoid"] = acs["state"] + acs["county"] + acs["tract"]

    out = pd.DataFrame({"geoid": acs["geoid"], "year": acs["year"]})

    # population / age
    out["pop_total"] = acs["B01001_001E"]
    out["pop_under5"] = acs["B01001_003E"] + acs["B01001_027E"]
    out["pop_under18"] = (

```

```

        acs["B01001_003E"] + acs["B01001_004E"] + acs["B01001_005E"] +
↪acs["B01001_006E"] +
        acs["B01001_027E"] + acs["B01001_028E"] + acs["B01001_029E"] +
↪acs["B01001_030E"]
    )
    out["share_under18"] = safe_div(out["pop_under18"], out["pop_total"])

# race shares
    out["share_white"] = safe_div(acs["B02001_002E"], acs["B02001_001E"])
    out["share_black"] = safe_div(acs["B02001_003E"], acs["B02001_001E"])
    out["share_asian"] = safe_div(acs["B02001_005E"], acs["B02001_001E"])
    out["share_two_plus"] = safe_div(acs["B02001_008E"], acs["B02001_001E"])

# education BA+
    ba_plus = acs["B15003_022E"] + acs["B15003_023E"] + acs["B15003_024E"] +
↪acs["B15003_025E"]
    out["ba_plus_share_25p"] = safe_div(ba_plus, acs["B15003_001E"])

# employment
    out["lfrp_16p"] = safe_div(acs["B23025_002E"], acs["B23025_001E"])
    out["unemp_rate"] = safe_div(acs["B23025_005E"], acs["B23025_003E"])

# commute under 35 minutes
    commute_under35 = (
        acs["B08303_002E"] + acs["B08303_003E"] + acs["B08303_004E"] +
↪acs["B08303_005E"] +
        acs["B08303_006E"] + acs["B08303_007E"] + acs["B08303_008E"]
    )
    out["commute_under35_share"] = safe_div(commute_under35, acs["B08303_001E"])

# internet subscription
    out["internet_sub_share"] = safe_div(acs["B28002_002E"], acs["B28002_001E"])

# affordable housing proxy (cost burden <30%)
    rent_affordable = (
        acs["B25070_002E"] + acs["B25070_003E"] + acs["B25070_004E"] +
↪acs["B25070_005E"] + acs["B25070_006E"]
    )
    owner_affordable = (
        (acs["B25091_003E"] + acs["B25091_004E"] + acs["B25091_005E"] +
↪acs["B25091_006E"] + acs["B25091_007E"]) +
        (acs["B25091_014E"] + acs["B25091_015E"] + acs["B25091_016E"] +
↪acs["B25091_017E"] + acs["B25091_018E"])
    )
    denom_hh = acs["B25070_001E"] + acs["B25091_001E"]

```

```

    out["affordable_housing_share"] = safe_div(rent_affordable +
↪owner_affordable, denom_hh)

    # female above poverty
    out["female_above_poverty_share"] = safe_div(acs["B17001_046E"],
↪(acs["B17001_017E"] + acs["B17001_046E"]))

    # early education enrollment proxy (3-4 enrolled / under5 pop)
    enrolled_3_4 = acs["B14003_004E"] + acs["B14003_013E"] + acs["B14003_032E"]
↪+ acs["B14003_041E"]
    out["early_ed_enroll_share"] = safe_div(enrolled_3_4, out["pop_under5"])

    # health insurance coverage (1 - uninsured/total)
    uninsured = (
        acs["B27001_005E"] + acs["B27001_008E"] + acs["B27001_011E"] +
↪acs["B27001_014E"] + acs["B27001_017E"] +
        acs["B27001_020E"] + acs["B27001_023E"] + acs["B27001_026E"] +
↪acs["B27001_029E"] +
        acs["B27001_033E"] + acs["B27001_036E"] + acs["B27001_039E"] +
↪acs["B27001_042E"] + acs["B27001_045E"] +
        acs["B27001_048E"] + acs["B27001_051E"] + acs["B27001_054E"] +
↪acs["B27001_057E"]
    )
    out["insured_share"] = 1.0 - safe_div(uninsured, acs["B27001_001E"])

    # gini + income + housing occupancy/value
    out["gini"] = acs["B19083_001E"]
    out["per_capita_income"] = acs["B19301_001E"]
    out["occupied_units"] = acs["B25002_002E"]
    out["median_home_value"] = acs["B25077_001E"]

    # growth features (year-over-year within tract)
    out = out.sort_values(["geoid", "year"])
    for col in ["per_capita_income", "occupied_units", "median_home_value"]:
        out[f"{col}_growth"] = out.groupby("geoid")[col].pct_change()

    return out

# =====
# 6) Merge ACS features with IGS
# =====
def merge_igs_acs(igs: pd.DataFrame, feats: pd.DataFrame) -> pd.DataFrame:
    merged = igs.merge(feats, on=["geoid", "year"], how="left")
    return merged

```

```

# =====
# 7) Run pipeline
# =====
# 7a) Load IGS

# igs_path = r"C:
↳ \Users\jabba\Desktop\Code\machine_learning\AUC_mastercard_challenge\src\Inclusive_Growth_Sc
↳ csv"

igs = load_igs_any(IGS_path)
igs.to_parquet(OUT_DIR / "igs_clean.parquet", index=False)

# 7b) Download ACS for needed years
acs = download_acs_all(sorted(igs["year"].unique().tolist()), CENSUS_API_KEY)
acs.to_parquet(OUT_DIR / "acs_raw.parquet", index=False)

# 7c) Feature engineering
feats = make_features(acs)
feats.to_parquet(OUT_DIR / "acs_features.parquet", index=False)

# 7d) Merge
model_df = merge_igs_acs(igs, feats)
model_df.to_parquet(OUT_DIR / "igs_x_acs.parquet", index=False)

print(model_df.shape)
print(model_df[["igs_total", "affordable_housing_share", "internet_sub_share", "insured_share"]].
↳ describe())

```

C:\Users\jabba\AppData\Local\Temp\ipykernel_17904\3170011791.py:44:
DtypeWarning: Columns (2,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,22,23,24,
25,26,27,28,29,30,31,32,33,34,35,36,37,38,39,40,41,42,43,44,45,46,47,48,49,50,51
,52,53,54,55,56,57,58,59,60,61,62,63,64,65,66,67,68,69,70,71) have mixed types.
Specify dtype option on import or set low_memory=False.
raw = pd.read_csv(p, header=None)

```

-----
TypeError                                Traceback (most recent call last)
File ~\anaconda3\Lib\site-packages\fastparquet\writer.py:257, in convert(data,
↳ se)
    254 if converted_type == parquet_thrift.ConvertedType.UTF8:
    255     # getattr for new pandas StringArray
    256     # TODO: to bytes in one step
--> 257     out = array_encode_utf8(data)
    258 elif converted_type == parquet_thrift.ConvertedType.DECIMAL:

```

```
File ~\anaconda3\Lib\site-packages\fastparquet\speedups.pyx:49, in fastparquet.  
↳speedups.array_encode_utf8()
```

TypeError: bad argument type for built-in operation

During handling of the above exception, another exception occurred:

ValueError Traceback (most recent call last)

Cell In[3], line 356

```
348 # =====  
349 # 7) Run pipeline  
350 # =====  
351 # 7a) Load IGS  
352  
353 # igs_path = r"C:  
↳\Users\jabba\Desktop\Code\machine_learning\AUC_mastercard_challenge\src\Inclusive_Growth_S  
↳csv"  
355 igs = load_igs_any(IGS_path)  
--> 356 igs.to_parquet(OUT_DIR / "igs_clean.parquet", index=False)  
358 # 7b) Download ACS for needed years  
359 acs = download_acs_all(sorted(igs["year"].unique().tolist()),  
↳CENSUS_API_KEY)
```

```
File ~\anaconda3\Lib\site-packages\pandas\util\_decorators.py:333, in  
↳deprecate_nonkeyword_arguments.<locals>.decorate.<locals>.wrapper(*args,  
↳**kwargs)
```

```
327 if len(args) > num_allow_args:  
328     warnings.warn(  
329         msg.format(arguments=_format_argument_list(allow_args)),  
330         FutureWarning,  
331         stacklevel=find_stack_level(),  
332     )  
--> 333 return func(*args, **kwargs)
```

```
File ~\anaconda3\Lib\site-packages\pandas\core\frame.py:3118, in DataFrame.  
↳to_parquet(self, path, engine, compression, index, partition_cols,  
↳storage_options, **kwargs)
```

```
3037 """  
3038 Write a DataFrame to the binary parquet format.  
3039 (...)  
3114 >>> content = f.read()  
3115 """  
3116 from pandas.io.parquet import to_parquet  
-> 3118 return to_parquet(  
3119     self,  
3120     path,  
3121     engine,
```

```

3122     compression=compression,
3123     index=index,
3124     partition_cols=partition_cols,
3125     storage_options=storage_options,
3126     **kwargs,
3127 )

```

File ~\anaconda3\Lib\site-packages\pandas\io\parquet.py:482, in to_parquet(df, path, engine, compression, index, storage_options, partition_cols, filesystem, **kwargs)

```

    478 impl = get_engine(engine)
    480 path_or_buf: FilePath | WriteBuffer[bytes] = io.BytesIO() if path is
↳ None else path
--> 482 impl.write(
    483     df,
    484     path_or_buf,
    485     compression=compression,
    486     index=index,
    487     partition_cols=partition_cols,
    488     storage_options=storage_options,
    489     filesystem=filesystem,
    490     **kwargs,
    491 )
    493 if path is None:
    494     assert isinstance(path_or_buf, io.BytesIO)

```

File ~\anaconda3\Lib\site-packages\pandas\io\parquet.py:351, in FastParquetImpl.write(self, df, path, compression, index, partition_cols, storage_options, filesystem, **kwargs)

```

    346     raise ValueError(
    347         "storage_options passed with file object or non-fsspec file pat."
    348     )
    350 with catch_warnings(record=True):
--> 351     self.api.write(
    352         path,
    353         df,
    354         compression=compression,
    355         write_index=index,
    356         partition_on=partition_cols,
    357         **kwargs,
    358     )

```

File ~\anaconda3\Lib\site-packages\fastparquet\writer.py:1340, in write(filename, data, row_group_offsets, compression, file_scheme, open_with, mkdirs, has_nulls, write_index, partition_on, fixed_text, append, object_encoding, times, custom_metadata, stats)

```

1336     fmd.key_value_metadata = kvm
1338 if file_scheme == 'simple':
1339     # Case 'simple'

```

```

-> 1340     write_simple(filename, data, fmd,
1341                     row_group_offsets=row_group_offsets,
1342                     compression=compression, open_with=open_with,
1343                     has_nulls=None, append=False, stats=stats)
1344 else:
1345     # Case 'hive', 'drill'
1346     write_multi(filename, data, fmd,
1347                 row_group_offsets=row_group_offsets,
1348                 compression=compression, file_scheme=file_scheme,
1349                 write_fmd=True, open_with=open_with,
1350                 makedirs=makedirs, partition_on=partition_on,
1351                 append=False, stats=stats)

```

```

File ~\anaconda3\Lib\site-packages\fastparquet\writer.py:1001, in
-> write_simple(fn, data, fmd, row_group_offsets, compression, open_with,
-> has_nulls, append, stats)
    999 of = open_with(fn, mode)
    1000 with of as f:
-> 1001     write_to_file(f)

```

```

File ~\anaconda3\Lib\site-packages\fastparquet\writer.py:985, in write_simple.
-> <locals>.write_to_file(f)
    983 rgs = fmd.row_groups
    984 for i, row_group in enumerate(data):
--> 985     rg = make_row_group(f, row_group, fmd.schema,
    986                         compression=compression, stats=stats)
    987     if rg is not None:
    988         rgs.append(rg)

```

```

File ~\anaconda3\Lib\site-packages\fastparquet\writer.py:802, in
-> make_row_group(f, data, schema, compression, stats)
    800     else:
    801         st = column.name in stats
--> 802         chunk = write_column(f, coldata, column,
    803                             compression=comp, stats=st)
    804         cols.append(chunk)
    805 rg = ThriftObject.from_fields(
    806     "RowGroup", num_rows=rows, columns=cols,
    807     total_byte_size=sum([c.meta_data.total_uncompressed_size for c in
-> cols]))

```

```

File ~\anaconda3\Lib\site-packages\fastparquet\writer.py:635, in write_column(f
-> data0, selement, compression, datapage_version, stats)
    631     data = data.astype('int32')
    633 if datapage_version == 1:
    634     bdata = b"".join([
--> 635         repetition_data, definition_data, encode[encoding](data,
-> selement), 8 * b'\x00'

```

```

636     ])
637     dph = parquet_thrift.DataPageHeader(
638         num_values=check_32(row_end - row_start),
639         encoding=getattr(parquet_thrift.Encoding, encoding),
640     (...))
641     i32=1
642     )
643     10 = len(bdata)

```

File ~\anaconda3\Lib\site-packages\fastparquet\writer.py:388, in

```

↳ encode_plain(data, se)
    386 def encode_plain(data, se):
    387     """PLAIN encoding; returns byte representation"""
--> 388     out = convert(data, se)
    389     if se.type == parquet_thrift.Type.BYTE_ARRAY:
    390         return pack_byte_array(list(out))

```

File ~\anaconda3\Lib\site-packages\fastparquet\writer.py:283, in convert(data, se)

```

    280     except Exception as e:
    281         ct = parquet_thrift.ConvertedType._VALUES_TO_NAMES[
    282             converted_type] if converted_type is not None else None
--> 283         raise ValueError('Error converting column "%s" to bytes using '
    284                             'encoding %s. Original error: '
    285                             '%s' % (data.name, ct, e))
    286 elif "str" in str(dtype):
    287     try:

```

ValueError: Error converting column "Place" to bytes using encoding UTF8.
↳ Original error: bad argument type for built-in operation

```

[ ]: print("IGS shape:", igs.shape)
print("Unique GEOIDs:", igs["geoid"].nunique())
print("Years:", sorted(igs["year"].unique()))
print(igs["geoid"].head())

# only if state/county columns exist in your cleaned IGS
for c in ["State", "County"]:
    if c in igs.columns:
        print(c, igs[c].nunique(), igs[c].value_counts().head(10))

print("ACS raw shape:", acs.shape)
print("ACS features shape:", feats.shape)
print("Merged shape:", model_df.shape)
print("Merged unique GEOIDs:", model_df["geoid"].nunique())

```

```

[ ]: 5f5152d426a2b4a34d8b5697a6d2b5d48527941cc5a1369b

```


[]: `http://127.0.0.1:8899/lab?token=5f5152d426a2b4a34d8b5697a6d2b5d48527941cc5a1369b`

[]:

[]: